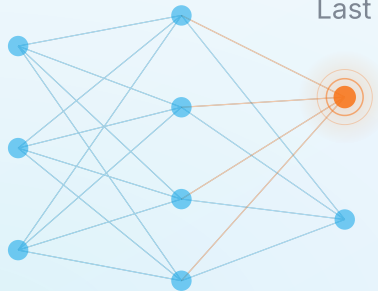


Effective Claude Code

Lessons from months of real sessions

The win isn't faster typing. It's orchestration.

Last 30 days alone: 63 sessions · 978 sub-agents dispatched · 31K tool calls · one person.



63 sessions dispatched 978 sub-agents — in one month

Last 30 days on disk (Jun 8 – Jul 9) · one person · real merged work on Fission, fission.io, the harness itself, and a blog.

63

sessions

978

sub-agents

31.4k

tool calls

434MB

of transcripts

60.7k

model turns

≈\$36k

all-time cost*

~15 sub-agents / session

978 of 1,041 transcripts on disk are sub-agents I dispatched — 512 direct Agent runs, the rest Workflow fan-outs.

That's the whole talk: orchestration, not autocomplete.

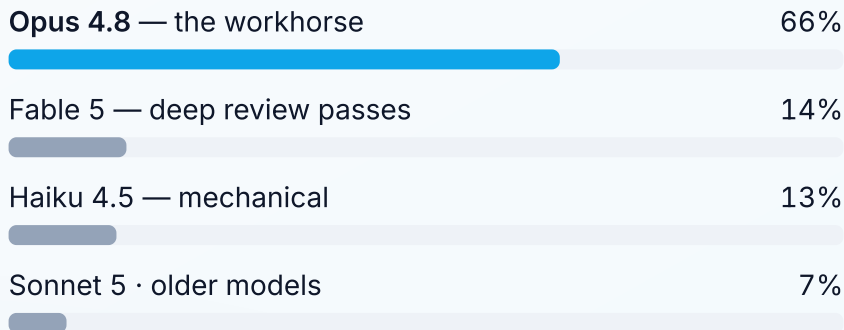
WHERE THE 31.4K CALLS WENT

Bash	15,578
Read	8,345
Edit	3,654
Agent — sub-agents	512

Deep-mined from the on-disk logs (transcript retention keeps ~30 days). *all-time cost = exact per-model, per-cache-tier list pricing over every logged token since the start (≈100M+ via [/usage](#)); ~63% of it is discounted cache-reads — fresh generation is only about a third.

Opus thinks, cheap models type

MODEL MIX · 60.7K ASSISTANT TURNS · LAST 30 DAYS



Opus + Fable for judgement (**80%** of turns); Haiku + Sonnet for the rote ~20%. Capacity matched to difficulty — **Lever 5**, quantified.



LONGEST SINGLE SESSIONS · LINES

12,149 · multi-tenancy cluster-watch

10,602 · feature debugging

9,452 · controller migration

9,135 · perf-enhancement RFC

By wall-clock, `/usage` clocks the longest single session at **31d 17h 43m** — resumed across weeks, not typed in one sitting.

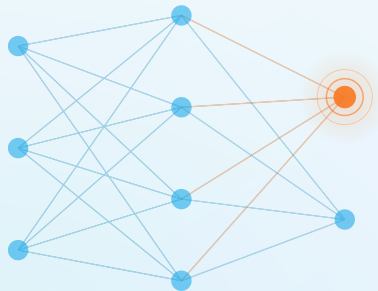
Max impact: the flagship **controller migration** threads through **209 transcripts**; the docs-site overhaul, a dedicated 91-transcript project. (*Stories, Part 2.*)

The one big idea

Treat Claude Code as an **orchestration system you direct** —
not an autocomplete you poke.

The highest-leverage sessions all had the same shape:

think → plan → fan out → verify



Five levers

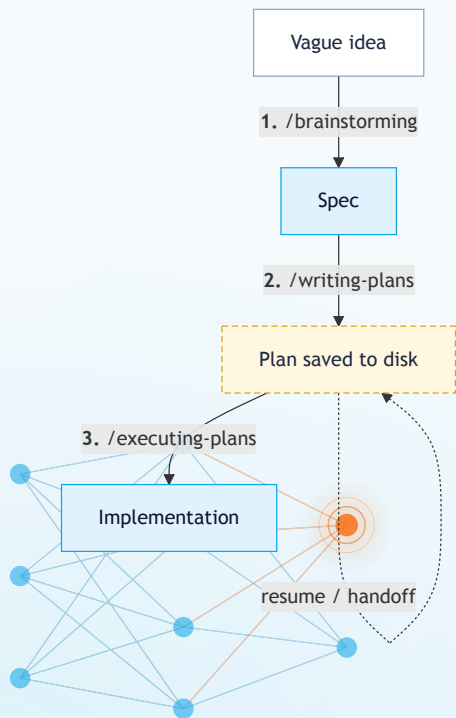
In order of impact

1. Plan-first workflow
2. Skills — stock and your own
3. Parallelism: worktrees + subagents
4. Config-as-code (CLAUDE.md)
5. Model & effort matching



Lever 1 — Plan first, code later

The sequence behind **every** big effort:



Plans are **saved to disk** — durable artifacts you can resume, review, and hand off.

"Building a comprehensive understanding of core components of fission — router, executor, CLI, builder, function pods... Prepare a list of ideas, think about the top 2-3, prepare a detailed plan for each. Use /brainstorming and /writing-plans as needed."

Always start in plan mode (read-only exploration) →
↑ **Tab** to **auto mode** once the plan is approved. Think first, execute hands-off second.

Principle: no blind coding. The plan is the contract.

A tight plan beats a long one

Mined from 36 real plans. Length $\neq$ quality — these four traits do the work.

Specificity over abstraction

File paths, line numbers, function signatures — not "update the webhook."

Reuse-by-reference

Point at an existing pattern — "mirror this file", "follow that checklist", "use the same idiom" — so execution is copy-adapt, not invent.

De-risk by phasing

Break into independently shippable PRs — a packaging change as 3 PRs, a controller migration as a foundation PR + 8 streams.

Verification $\approx$ 40% of the plan

build/vet/lint/race, codegen-drift, unit vs envtest vs integration, backward-compat guards.



Realistic scope wins: the poolmgr coverage plan targeted 54.6% $\rightarrow$ ~70%, explicitly *not* 100% — and named which lines only integration tests can reach.

Lever 2 — The skill stack that paid off

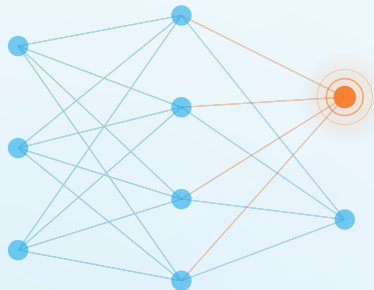
PROCESS SKILLS (RIGID — USE FOR DISCIPLINE)

- | **brainstorming** — vague idea → spec
- | **writing-plans / executing-plans** — the backbone
- | **systematic-debugging** — root-cause before patch
- | **verification-before-completion** — evidence before "done"

COLLABORATION & ISOLATION

- | **subagent-driven-development** — fan a plan to parallel agents
- | **using-git-worktrees** — isolated parallel branches
- | **requesting / receiving-code-review** — pre-merge rigor

Principle: skills are *rigid process where you want discipline* (debugging, TDD, verification) and *flexible guidance* everywhere else. Knowing the concept ≠ using the skill — invoke it.



Build your own skill library

Repeat a workflow 3+ times → codify it. 56 skills · 5 suites, one source of truth feeding Claude Code (2 profiles) + the `pi` agent.

CI & Go

debug-ci → watch-ci
go-deps-security-sweep
improve-codecov-coverage
analyze-go-pprof · analyze-prometheus-tsdb

PR review & quality

make-pr-easy-to-review
resolve-bot-review-threads
pr-review-canvas · deslop

Security & maintenance

author-security-advisory
remediate-codeql-alerts
source-code-for-gh-advisory
triage-gh-backlog · local SQLite mirror

Docs / Hugo site

write-hugo-blog-post
author-mermaid-diagram
verify-hugo-build · bump-hugo-versions
generate-og-images · audit-static-site

Beyond engineering — daily-work automation in real Chrome, via `agent-browser`

approve-workday-tasks · fill-workday-timesheet · apply-workday-leave · record-engage-activity · list-week-meetings — all on one login-microsoft-sso building block, every one **review-first**: nothing submits without my confirm.

And I didn't hand-curate all of this — the harness proposed it. ↓

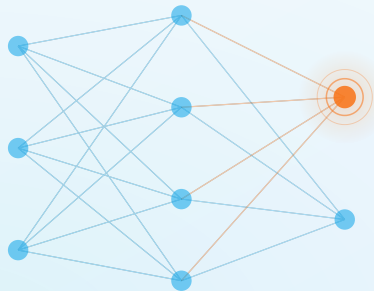
The harness that improves itself

/HARVEST-AUTOMATION

Mines past sessions for **recurring** patterns (≥ 2 occurrences, evidence required — session id + verbatim quote), then routes each to the right home:

- | multi-step workflow → **a new skill**
- | repeated correction → **CLAUDE.md** fact
- | stated preference → **memory**
- | repeated approval → **permission allowlist**

Replaced the old `retrospect` + `workflow-from-chats`. Authoring is delegated to `writing-skills` — compose, don't reimplement.



WHY THE LIBRARY COMPOUNDS

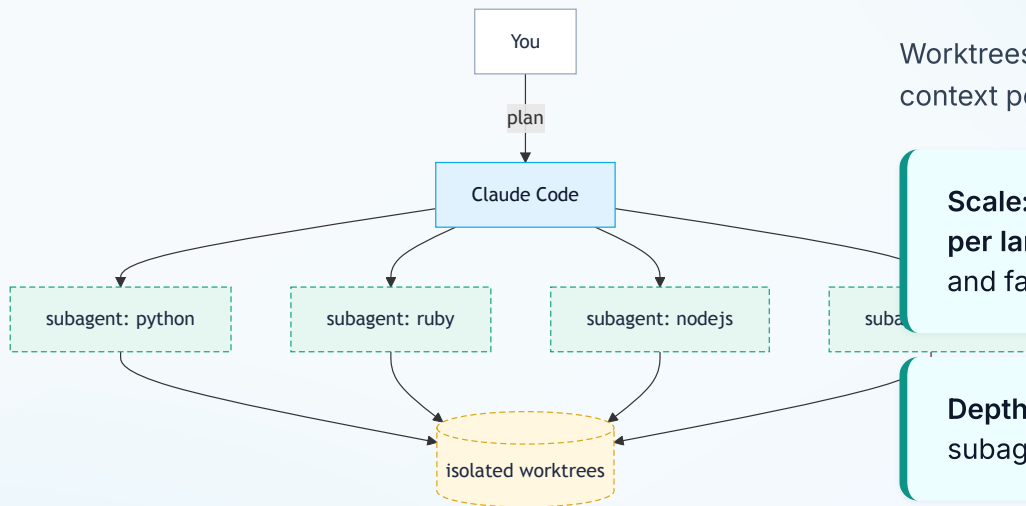
Atomic + chainable — `debug-ci` → `watch-ci` → `resolve-bot-review-threads`: the fix → push → resolve → re-request → watch loop.

One source, many harnesses — a single `skills/` tree feeds Claude Code (2 profiles) + `pi`, symlinked. Edit once, everywhere.

Provenance, enforced — every skill carries a `.source.json` sidecar; the catalog is *generated* from them and `make skills-doctor` fails on drift.

Suites — skills packaged as shareable per-domain products with a one-line install: `second-brain` · `pr-shepherding` · `oss-maintainer` · `go-ci-health` · `hugo-site`.

Lever 3 — Worktrees + parallel subagents

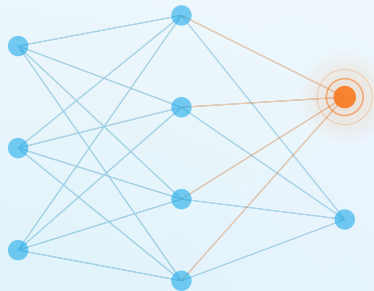


Worktrees give each line of work its **own workspace** → no context pollution, no merge collisions, real concurrency.

Scale: the environments bulk update ran **one subagent per language**, each independently reading code, editing, and fact-checking.

Depth: RFC0006 fixes ran in a dedicated worktree, each subagent owning one fix end-to-end.

Rule: parallelism is only safe when units are **independent**. Decompose first.



Lever 4 — CLAUDE.md as config-as-code

GLOBAL (UNIVERSAL)

- NEVER publish secrets / commit `.env`
- Markdown **one sentence per line** — renders identically, makes git diffs surgical

PER-PROJECT (THE FACTS A COLLABORATOR NEEDS)

- Fission: make targets, "never hand-edit generated code", router public/internal split post-GHSA, testify + `t.Parallel()`
- Blog / fission.io: Hugo theme, frontmatter fields, version shortcodes

Tier the knowledge: core, always-relevant facts in `CLAUDE.md` ; deep or rarely-needed detail in `.claude/resources/*.md` , pulled in only when a task calls for it — so the always-on context stays lean. (*fission.io keeps its mermaid palette + diagram rules in `.claude/resources/mermaid-diagrams.md` .*)

Keep global minimal & universal · push specifics into per-project files & resources



Lever 5 — Model & effort matching

Work	Model
Architecture, planning, review	Opus 4.8
Overflow / fallback	Opus 4.7
Mechanical, verification	Fable 5 / Haiku
Hardest problems	bump /effort

Switch between tasks, not mid-thought. Use Opus for the brainstorm + plan, then `/model` down to a cheaper model for known-good mechanical execution.

Fast mode (Opus, faster output) for interactive iteration.

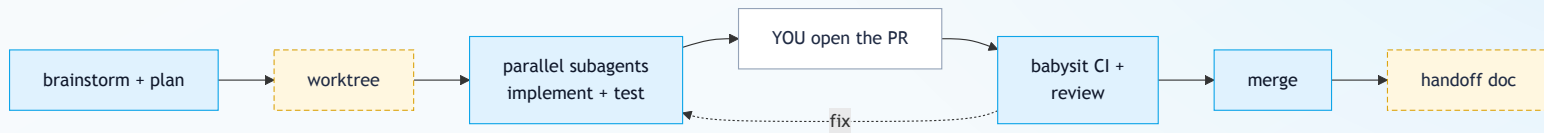
No model thrashing — assess complexity up front; don't retry the same problem across models.

Different vendors, if you have them. Models from different labs have different blind spots — a second-opinion or cross-review pass catches what one family won't.



Don't pay Opus prices for rote edits · don't hand architecture to a small model

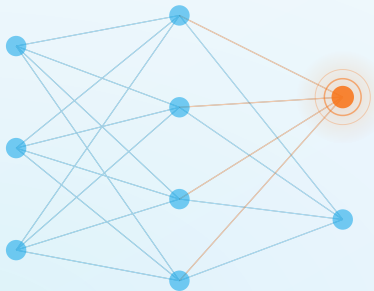
The macro-loop — plan to merge



"Please babysit PR until it's green and achieves objectives for merge."

"Consolidate changes in one branch. Avoid spreading across PRs."

Principle: give **terminal conditions** ("until green") and **constraints** ("one branch") so the agent runs autonomously without drifting. Clear human/agent boundary: *agent drives CI, you merge.*



Set a goal — wake up to a PR

The highest-leverage pattern I run: kick off an RFC at night, review a green PR in the morning.

ONE PROMPT SETS THE TERMINAL CONDITION

```
/goal Implement RFC-00XX end-to-end.  
Run the full review battery as needed,  
get CI green, leave the PR ready to merge.  
Don't stop until it's ready — or you're  
genuinely blocked.
```

"I am going to sleep, you can continue till possible, fix failing tests and then continue. Do not stop unless you are very unsure."

Overnight · unattended · one branch. By morning: an implemented, self-reviewed, CI-green PR waiting on *my* merge.

Why it works: a **goal** is a terminal condition ("PR ready to merge") plus a quality bar ("review battery, CI green"). The agent loops until both are met.

What keeps it alive: **task notifications** re-invoke the agent when CI finishes — no polling, no context reset; a **handoff doc** records status + gotchas so the next session picks up cleanly.

Guardrails: plan-mode first (so the approach is sound), one branch, human merges. The agent owns the grind; you own the gate.

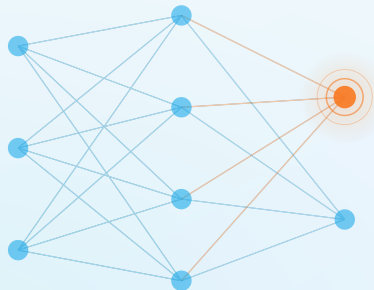
Only for decomposable, verifiable work — tests + CI are the oracle that makes an unattended run safe.

The review battery — many lenses, one PR

Big changes get more than one reviewer. Each tool is a specialized lens; run them together.

THE BATTERY I INVOKE ON BIG PRS

- `/code-review` — correctness & design
- `/security-review` — vulns & data-flow
- `/simplify` — cut needless complexity
- `/thermo-nuclear-code-quality-review` — strict maintainability
- `/deslop` — strip AI slop
- `/make-pr-easy-to-review` — tidy for humans



`/pr-review-canvas` for the *manual* pass — an interactive HTML walkthrough of the diff that surfaces the **high-impact areas first**, so human attention lands where it matters.

▶ live demo

router perf PR #3493 → `demo/pr-review-3493.html`

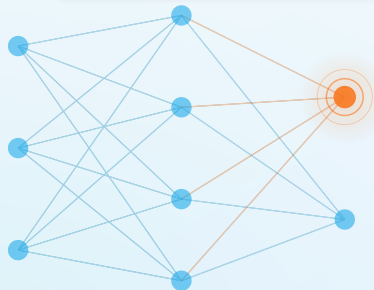
Principle: no single reviewer catches everything. A battery of narrow, opinionated lenses beats one generic pass — and it runs *inside* the overnight goal automatically.

Loops — the pattern behind all of it

A **loop** = a reusable agent workflow with a **stopping condition** + **verification gates**. "Babysit until green", "migrate every test", "the overnight goal" — all loops.

The deck's whole spine — **terminal conditions + verify with evidence** — is really just: *write good loops*.

We've run loops for the **test migration** (per-test), the **reconciler streams**, a **docs sweep**, and the **review battery**.



A WHOLE LIBRARY OF THEM

forwardfuture.com/loop-library — ~70 named loops across **Engineering · Evaluation · Operations · Content · Design**.

"The docs sweep" — keep docs current

"The sub-50 ms page-load loop"

"The full product evaluation loop"

"The self-improving champion loop"

Steal a loop, adapt its checks + stop condition, run it.

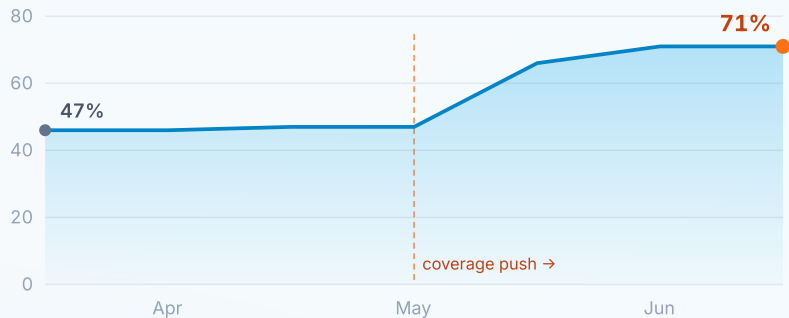
Stories

What it looked like in practice

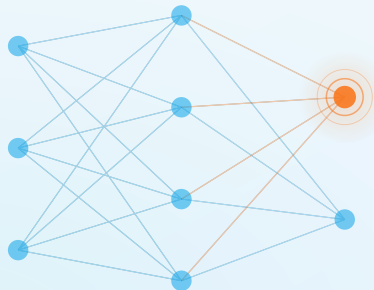


Story 1 — first tests, then coverage

COVERAGE ON **MAIN** · 3-MONTH TREND



70.97% covered · 18.3k / 25.8k lines **+24 pts** in 3 months

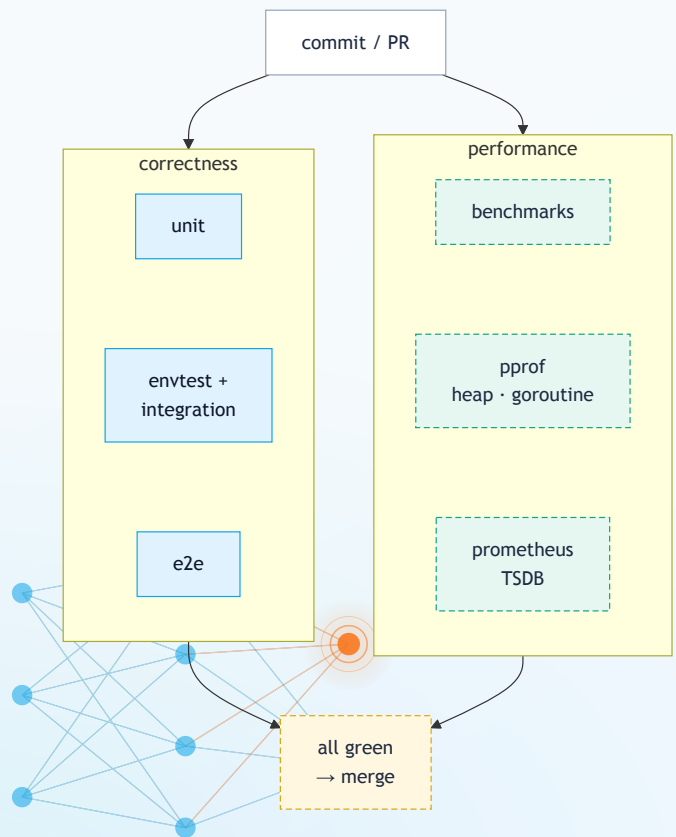


1 · Migrate — ~17k lines of bash integration tests → a Go framework. Long autonomous runs, one test at a time, clear stop condition.

2 · Coverage first — *then* drove **main** from **47%** → **71%**. Build the safety net before the features.

3 · Ship faster — with the net in place, the RFC features (Stories 2–3) landed **quickly & with fewer regressions**.

Every change runs a gauntlet

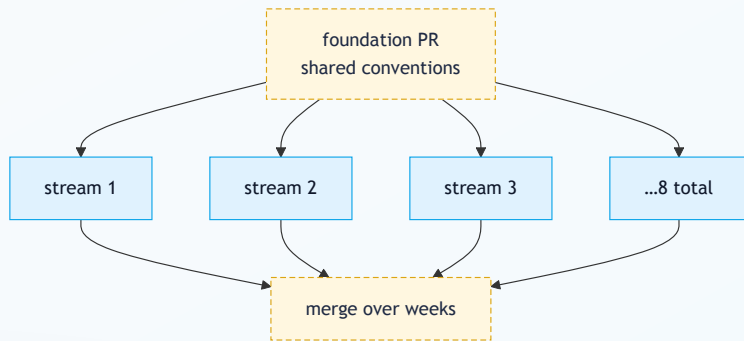


Correctness — $\text{unit (fast)} \cdot \text{envtest} + \text{integration} \cdot \text{e2e}$.
The integration layer catches what unit + e2e miss (cache-scoping, RBAC 30s timeouts).

Performance — benchmarks gate a regression (a real in-tree benchmark), `pprof` separates a leak from baseline, a `prometheus` TSDB shows over-the-run trends.

The gate that matters is the one your other layers are blind to. Run the full battery, not a subset.

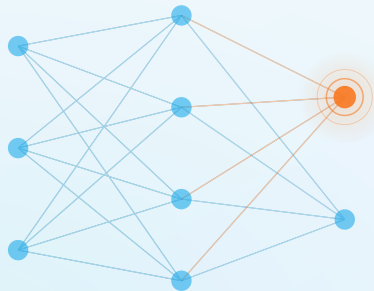
Story 2 — foundation first, then fan out



The task: a large migration touching every subsystem — ideal for parallelism, but only if the streams can't collide.

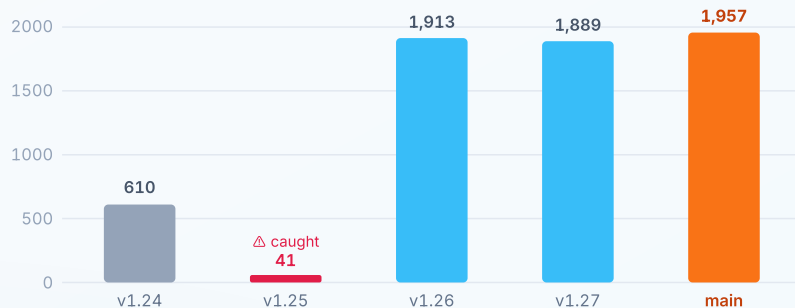
The pattern: a **foundation PR** fixes the shared conventions (interfaces, helpers, one predicate) up front — *then* the independent streams fan out and merge without conflicts.

Some bugs only integration catches: a missing permission caused a 30s timeout while unit + e2e stayed green — captured in the handoff doc.

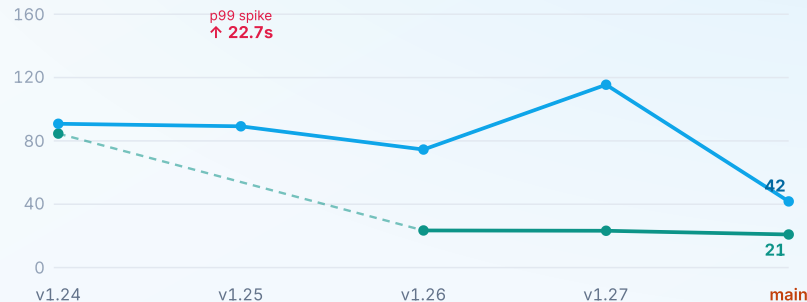


Story 3 — 3.2× throughput, measured every release

THROUGHPUT · RPS · ↑ HIGHER IS BETTER



LATENCY · MS · ↓ LOWER IS BETTER · COLD P50 WARM P99



3.2×

throughput
610→1,957

4×

lower p99
85→21 ms

2.2×

faster cold
91→42 ms

Sustained, not a one-shot — successive perf RFCs compounded across releases, all AI-driven & measured on every run.

The suite earns its keep — it caught the v1.25 warm-path collapse (41 rps), recovered by v1.26.

RFC-0020 `fission-benchmark` suite · real kind cluster · smoke, 1 sample/version · "main" = latest run.

Story 4 — the same playbook moves content

FISSION.IO DOCS OVERHAUL · ONE SESSION

"Holistically improve documentation based on the latest version of Fission. Make it lucid, readable and appealing. Use diagrams, mermaid diagrams wherever possible... Use dynamic workflows to work in parallel if required."

That one session fanned out **dozens of subagents and workflows** — multi-page rewrites, diagram generation, and a consistent visual theme, all at once.

The mermaid color-kit in *this* deck comes from that overhaul's conventions. The work compounds.

Principle: plan → fan out → verify with evidence works on *content* too — the evidence is a clean build, a rendered page, a green SEO audit.

THE REPEATABLE CONTENT PLAYBOOK

Diagrams that render right — `author-mermaid-diagram` : fits a narrow column, semantic palette.

Verify the live site — `agent-browser` loads real pages, screenshots, tunes layout against what actually renders.

SEO & reachability — `audit-static-site` · `report-site-analytics` (GA4 + GSC) · `add-llms-txt` · `generate-og-images` .

Authoring — `write-hugo-blog-post` · `verify-hugo-build` before publish.

Beyond code — same playbook, higher stakes

Two recent jobs, no code in either. The orchestration playbook — pointed at my taxes and my reading life.

AUDITED MY CA'S TAX RETURN

Claude **recomputed every rupee** against Form 16 + AIS and reproduced the CA's total *exactly* — proving the dispute was pure **classification**, not math.

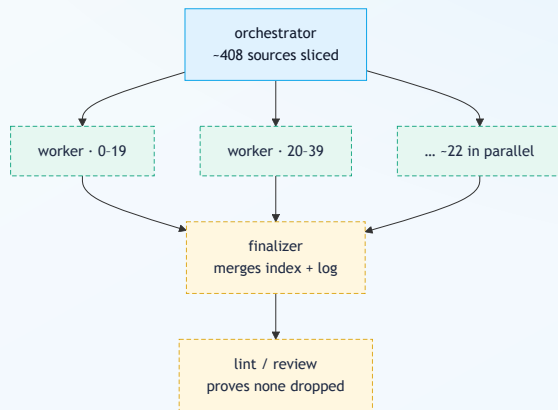
The **AIS SFT-18 code** (equity fund, STT -paid) settled it: gains mis-parked at slab rate belonged in **§111A/112A** — a real error the CA missed, in **two** returns.

Then it drafted the correction email and saved a **revisit checklist** to re-check the CA's revised numbers.

Verify with evidence — zero code written.

Same spine, further out: plan → fan out → verify holds on personal finance and personal knowledge — not just code.

FIRED READWISE → A KARPATY-STYLE LLM - WIKI



Each worker owns **only its slice** — forbidden to touch the shared `index` / `log`, so ~22 agents never collide. Six skills I built power it: ingest · query · lint · review · sync.

Working with the harness

Prompting, hygiene, and the traps



PART 3

How to prompt — 4 patterns that worked

Context-rich, constraints-forward

Scope & boundaries up front.

"We use Hugo in the blog, so adjust the post accordingly."

Autonomy within guardrails

Freedom + a stopping condition.

"Continue till possible... do not stop unless very unsure."

Numbered, surgical feedback

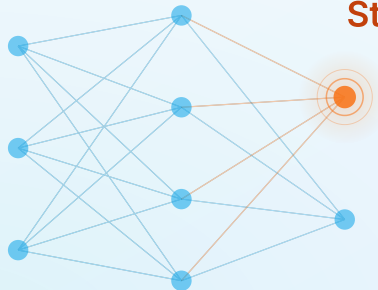
A checklist beats vague dissatisfaction.

*"1. don't pin the model name 2. paraphrase 3. avoid first person
4. refer to the PR over commits."*

Reframe scope, don't reject

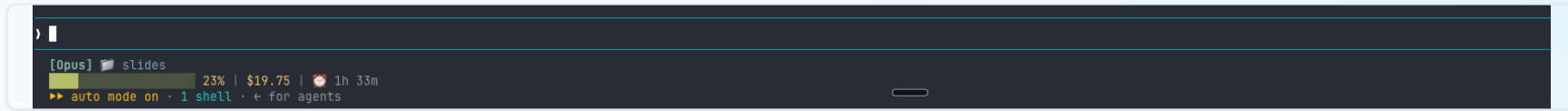
Usually the *scope* was wrong, not the work.

"It's not only about one fix... it should cover the wider migration."



Steer with constraints & reframing — not after-the-fact correction.

Session hygiene & measuring the harness



```
> [Opus] sLides 23% | $19.75 | 1h 33m  
▶ auto mode on · 1 shell · ← for agents
```

A custom statusline keeps a long run legible — model · folder · context % · \$cost · elapsed · mode.

Context: `/compact` + `autocompact` keep long auto-mode sessions alive; `/resume` across days; `/init` to seed CLAUDE.md.

Modes: start in `plan` (read-only), `↑Tab` to `auto` once approved — hands-off execution.

Fewer interrupts: a permission `allowlist` (go vet/lint, govulncheck, helm template, context7) + `/fewer-permission-prompts`.

CC-PROXY — SEE WHAT CLAUDE CODE ACTUALLY SENDS

`CLAUDE_PROXY=1` routes the session through a **zero-latency logging proxy**; every request tees to Markdown + SQLite, with Grafana dashboards for **tokens · top tools by bytes · cache hit rate · cost/hour**.

The "cut list" dashboard shows which tools bloat your context.

Principle: you can't tune what you can't see — measure the harness, then harvest.

Anti-patterns we actually hit

CI flake loops

Distinguish signal from noise. Re-run known flakes — don't "fix" them blindly.

RFC scope creep

Caught by phasing + explicit "defer to next release" calls.

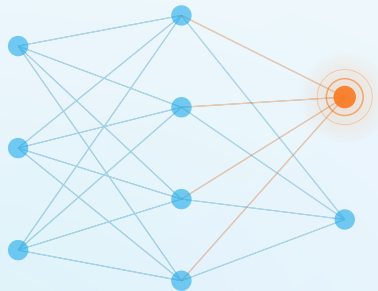
Silent verification gaps

Cache-scoping bugs invisible to unit + e2e — only integration caught them. Run the *full* suite, not a subset.

Codegen drift

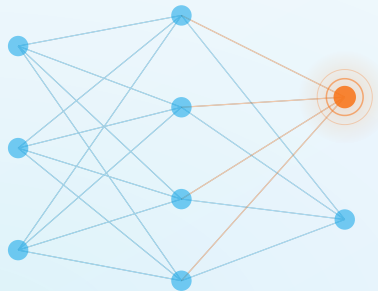
Regenerate, don't hand-edit generated files.

Verify with evidence, not optimism — name the gaps your tests can't see.



Top takeaways

1. Plan first; save the plan to disk.
2. Decompose into independent units; fan out.
3. Set a goal: terminal condition + a quality bar; loop until met.
4. Encode lessons in CLAUDE.md (+ resources) & your own skills.
5. Match model & effort to difficulty.
6. Steer with constraints & numbered feedback.
7. Verify with a review battery + evidence; write handoffs.
8. Measure, harvest & tune the harness; it compounds.



Orchestrate, don't autocomplete.

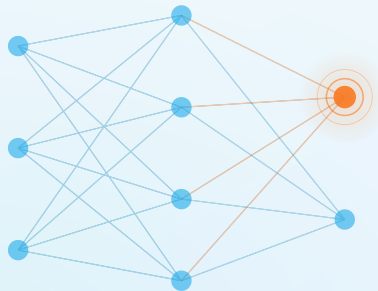
Steal this setup

Config-as-code: symlinked global + per-project CLAUDE.md, **56 skills in 5 installable suites**, a permission allowlist, a statusline, `cc-proxy` observability — one source of truth for Claude Code + `pi`.

Plan-first · fan-out with worktrees · verify with evidence · harvest & repeat.

The library reaches far past code — `debug-ci` · `pr-review-canvas` · `write-hugo-blog-post` · `itr-india` · `second-brain-ingest` · `fill-workday-timesheet`.

It's public → github.com/sanketsudake/harness-configs



Thank you

Questions?